Comparative study of performance of parallel Alpha Beta Pruning for different architectures

Shubhendra Pal Singhal
Department of Computer Science and Engineering
National Institute of Technology, Tiruchirappalli
shubhendrapalsinghal@gmail.com

M. Sridevi
Department of Computer Science and Engineering
National Institute of Technology, Tiruchirappalli
msridevi@nitt.edu

***Abstract*—Optimization of searching the best possible action depending on various states like state of environment, system goal etc. has been a major area of study in computer systems. In any search algorithm, searching best possible solution from the pool of every possibility known can lead to the construction of the whole state search space popularly called as minimax algorithm. This may lead to a impractical time complexities which may not be suitable for real time searching operations. One of the practical solution for the reduction in computational time is Alpha Beta pruning. Instead of searching for the whole state space, we prune the unnecessary branches, which helps reduce the time by significant amount. This paper focuses on the various possible implementations of the Alpha Beta pruning algorithms and gives an insight of what algorithm can be used for parallelism. Various studies have been conducted on how to make Alpha Beta pruning faster. Parallelizing Alpha Beta pruning for the GPUs specific architectures like mesh(CUDA) etc. or shared memory model(OpenMP) helps in the reduction of the computational time. This paper studies the comparison between sequential and different parallel forms of Alpha Beta pruning and their respective efficiency for the chess game as an application.**

***Index Terms*—Parallel algorithms, Minimax, Alpha Beta pruning, CUDA, OpenMP, Mesh architecture, Shared memory model**

I. Introduction

Playing a game strategically requires an individual to foresee all kinds of winning possibilities. The grading policy applied to game tree is generally +1 for winning and -1 for losing which ultimately helps the agent decide the next move. This may require the construction of whole state space implying that every possibility or action needs to be considered and whatever suits the best or takes the agent close to its goal, should be opted. Now, this is the general brute force method which might work practically for simpler and smaller games like Tic-Tac-toe, but the complex games like checkers or chess on 8*8 board, has a gigantic state space and searching using the brute force method in such a huge space is impractical. This gives us the motivation to study a better and feasible method called Alpha Beta pruning. The cases which turns out to be futile at the start of the search are rejected, thus reducing the state space for every move by significant amount. Furthermore, the efficiency of the Alpha Beta pruning can be further improved, to suit feasibility of running AI applications [1] consisting of accrued state space. Parallelism turns out to be one of the factors which can be used to improve the performance. This paper compares the performances and speedup obtained from various implementations of the parallel forms of Alpha Beta pruning on different architectures.

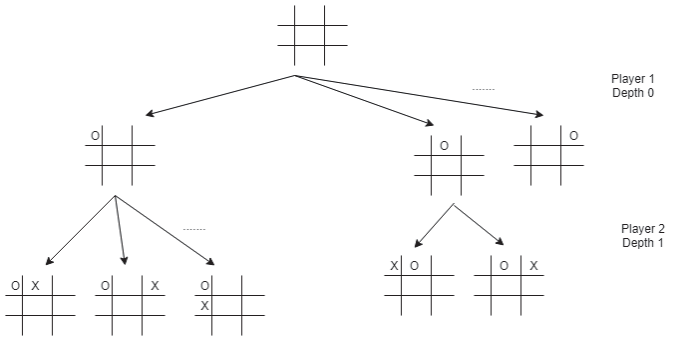

Fig. 1: Game tree for Tic-Tac-toe

The earlier the branches are pruned, the better is the efficiency of Alpha Beta pruning. Different architectures prune the branches at different time stamps, thus differing in the computation time.

A. Minimax Algorithm

Brute force search in the state space is the minimax algorithm. Minimax is a decision-making algorithm [2]. The main aim of the algorithm is to find the next best move as shown in Fig1 for the game Tic-Tac-Toe [3] (application of minimax) .

In the applications of Minimax algorithm (involving two players), first player is maximizer, and the second player is the minimizer. The evaluation score is assigned to the game board, where the maximizer and minimizer aims to get the highest score, and the lowest score possible respectively [2]. The implementation of the same in shown in the following algorithm 1 [4].

If every child branch is allocated to one processor and run the same algorithm in parallel for every child branch, then the parallel minimax is formed. The parent child will collect the answer from all the child branches and further

978-1-7281-4392-7/19/$31.00 ©2019 IEEE

Authorized licensed use limited to: Cornell University Library. Downloaded on May 13,2026 at 21:40:59 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Minimax Algorithm

```
function minimax(node,depth, maximizingPlayer)
if depth = 0 or node is a leaf node then
    return heuristic value of node
end
if maximizingPlayer then
    value = -∞
    while every child of node do
        value = max(value, minimax(child, depth-1,
          FALSE)
    end
    return value
end
else
    value = +∞
    while every child of node do
        value = min(value, minimax(child, depth-1,
          TRUE)
    end
    return value
end
```

propagate the optimal one to its ancestors till it finally converges to one parent i.e. root of the tree.

B. Alpha Beta Pruning

AlphaBeta pruning is a search algorithm that seeks to decrease the number of nodes that are evaluated by the minimax algorithm in its search tree. It is an adversarial search algorithm which stops evaluating a move when at least one possibility has been found that proves the move to be worse than a previously examined one. Such moves need not be considered further. In this way, this leads to pruning of the branch as this will never affect the final decision that agent has to make to achieve the goal.

1) Sequential Alpha Beta Pruning: The Alpha-Beta pruning algorithm emerged as an improvement over the usual Minimax algorithm. Both of these algorithms find out the best available move to the player and will return the exact utility regarded with that move. The execution time of Alpha-Beta pruning algorithm is comparatively faster than Minimax algorithm as they cut down(prune) the branches, thereby reducing the exploration space. The reason being, that the values calculated from these branches would not affect the final result. Since, time is not spent on exploring the other branches, these algorithms effectively abates the execution time. The implementation of the same in shown in the following algorithm 2, with an illustration shown in Fig 2.

2) Parallel Alpha Beta Pruning: Parallelizing Alpha Beta pruning can further reduce the execution time and improves the performance. The insight to the solution is to parallelize the searching of branches which are to be pruned i.e. the evaluation of the game tree is parallelized. Essentially, each branch of the game tree can be evaluated in parallel. Thus, the Alpha and Beta values are propagated at once to each of the node at the first level and the minimizing and corresponding

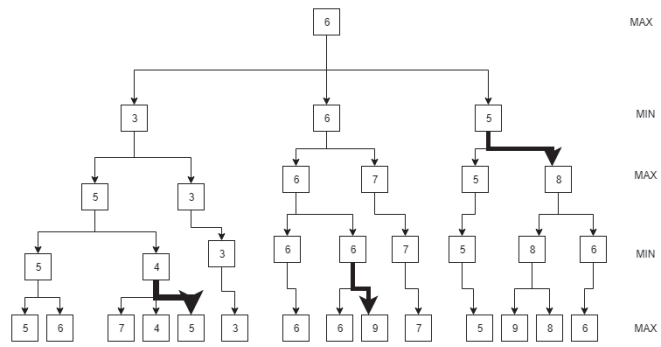

Fig. 2: Illustrating the sequential Alpha-Beta pruning

The highlighted arrows represents the cut that was being unnecessarily being explored in Minimax. This Alpha-Beta cut ensures that the branch is not explored further, because it is a mere waste of computation time. The highlighted arrows represents those branches which will not be further explored by Alpha-Beta Pruning algorithm, thus making it more efficient as it is evident from the last cut which saves up massive amount of time.

maximizing moves are evaluated simultaneously for all the branches. The final result of all the branches is propagated and is handled by the parent and the best move is computed with respect to the Alpha and Beta values of each branch.

Algorithm 2: Sequential Alpha Beta Pruning Algorithm

```
function ALPHA-BETA(node, depth, α, β,
  maximizingPlayer)
if depth = 0 or node is a leaf node then
    return heuristic value of node
end
if maximizingPlayer then
    value = -∞
    while every child of node do
        value = max(value, ALPHA-BETA(child,
          depth-1,α, β, FALSE)
        α = max(α, value)
        if β ≤ α then
            break
        end
    end
    return value
end
else
    value = +∞
    while every child of node do
        value = min(value, ALPHA-BETA(child,
          depth-1,α, β, TRUE)
        β = min(β, value)
        if β ≤ α then
            break
        end
    end
    return value
end
```

The partitioning of the tree for parallel computation is done on a per-child basis [5]. Each child of the tree evaluates the minimum and maximum moves together which is

Authorized licensed use limited to: Cornell University Library. Downloaded on May 13,2026 at 21:40:59 UTC from IEEE Xplore. Restrictions apply.

independent of the other branches(children of same parent) and thus computation of every branch is relatively faster [5]. After this computation is done, the parent decides upon the final result. The implementation of the same is given in algorithm 3.

Algorithm 3: Parallel Alpha Beta Pruning Algorithm

```
function Parallel-ALPHA-BETA(node, depth, α, β,
  maximizingPlayer)
if depth = 0 or node is a leaf node then
  | return heuristic value of node
end
if maximizingPlayer then
  | value = -∞
  | while every child of node in parallel do
  |   | value = max(value, Parallel-ALPHA-BETA(child,
  |   |   depth-1,α, β, FALSE)
  |   | α = max(α, value)
  |   | if β ≤ α then
  |   |   | break
  |   | end
  | end
  | return value
end
else
  | value = +∞
  | while every child of node in parallel do
  |   | value = min(value, Parallel-ALPHA-BETA(child,
  |   |   depth-1,α, β, TRUE)
  |   | β = min(β, value)
  |   | if β ≤ α then
  |   |   | break
  |   | end
  | end
  | return value
end
```

Over past few decades, proposal of implementing parallel Alpha Beta pruning on different architectures have been proposed [5]–[8]. One amongst them is the traditional algorithm which uses the prioritizing scheme for improving further efficiency. Principal-Variation algorithm [9] is one such attempts, which states the prioritizing scheme as the rule of searching the first branch at a PV node before the search of remaining branches begins. Since the implementation of PV Split algorithm on the existing architecture, there has been various possible optimization techniques [5] that have been suggested based on using that prioritizing scheme carefully and efficiently. All these schemes have been proposed and are being used in different environments like beam search, reordering being used in chess AI inbuilt Windows game [10]. But the real question that is never answered is that which method is the optimal under the specified constraints? So, this paper attempts to implement different parallel models for Alpha Beta pruning and also compare the results to find the optimal solution for the problem.

II. Experimental setup and implementation

There are various platforms available which allows the parallel implementation of an algorithm. Shared memory model and architectures like mesh have been used for the comparison. All the algorithms have been implemented for application of two player game chess for different board sizes [3].

A. Prerequisites for the implementation

For implementing the algorithm, the following setup is required.

1. A computing machine with 8GB RAM and Linux environment.
2. Installation of OpenMP.
3. Installation of NVIDIA CUDA Toolkit for Linux environment.

B. Algorithm used for implementation of Parallel Alpha Beta pruning

The way, parallelism is introduced in the Alpha Beta algorithm is by developing the concurrent processing program of multiple child nodes at every level of the minimax tree [11]. Amongst all the moves, if the best move can be evaluated first, then the rest of the moves can be straightaway ignored. Unfortunately, the a priori quality of estimation might supercede the time taken by sequential Alpha Beta pruning. The basis for our parallel Alpha-Beta implementation is the root splitting algorithm [9]. The main idea of this algorithm is to ensure that each node except for the root, is allocated to only one processor. To keep the effect of the Alpha Beta pruning, we split the children nodes of the root into clusters, each cluster being served by only one processor. Each child is then processed as the root of a sub search tree that will be visited in a sequential fashion. When a child finishes computing it returns its result to the root node, and the root node decides the best result. The mutual exclusion and the critical section constraints are taken into consideration while implementation. Along with root splitting, one another optimization is included. Reordering the children of a given node in a way to start exploring the most promising branches, thus pruning of the unwanted branches earlier making the algorithm more faster. Further, this algorithm is compared to one of the optimization i.e. beam search [10] which uses breadth-first search to construct a search tree. At every level of the search tree, it produces all the successors of the states at the current level and then sorts them in increasing order of their heuristic costs. The algorithm only explores a subset of those child nodes, by ignoring the least promising ones (according to the estimation function), and thus making the search faster.

Authorized licensed use limited to: Cornell University Library. Downloaded on May 13,2026 at 21:40:59 UTC from IEEE Xplore. Restrictions apply.

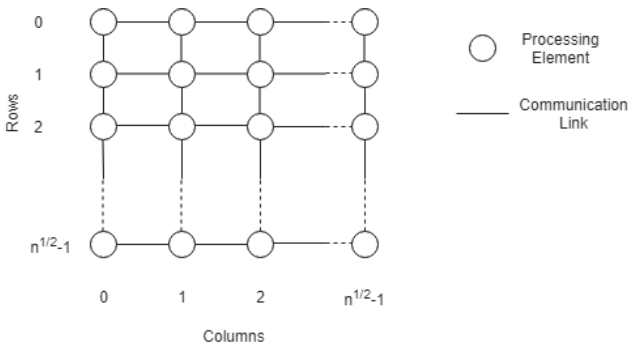

Fig. 3: Mesh Architecture of size n

C. Mesh Architecture - CUDA

The mesh architecture [12] of size n is a machine with n simple processors arranged in a square lattice. Processor P(i,j) represents the processor in row i and column j in mesh as shown in Fig3.

CUDA is a parallel computing framework created by Nvidia. The CUDA platform is a software layer that gives direct access to the virtual instruction set and parallel computational elements, for the execution of compute kernels. It is used for the implementation of parallel Alpha Beta pruning on mesh architecture.

The parallel architecture of the Nvidia GPUs is made up of a multiple sets of pipelined multiprocessors. The parallel computation on the GPU is performed in a similar fashion, as a set of concurrently executing thread blocks, which are organized into a 1D or 2D grid [7].

1) Implementation: In CUDA, the parallel processing of child nodes is done using multiple thread blocks(mesh architecture).

The parts of the root split algorithm executed in parallel by all of the processors are node evaluation and optimal move generation. [6], [7].

D. Shared Memory Model - OpenMP

Shared memory system has single (shared) space accessible by multiple processors but each process has its own address space which is not accessible by other processes. [13]

OpenMP (Open Multi-Processing) is an application programming interface (API) that supports multi-platform shared memory multiprocessing programming.

It is a framework with the model of parallel programming which runs on a computer cluster. OpenMP is used for parallelism within a multi-core node. It is used for the implementation of shared memory model of parallel Alpha Beta pruning.

1) Implementation: Every part of root split algorithm that is intended to keep running in parallel is assigned a thread from the pool of forerun created free threads. Every thread assigned is in the ”running” state which propagates the result to the parent thread which ultimately decides that which branch is optimal.

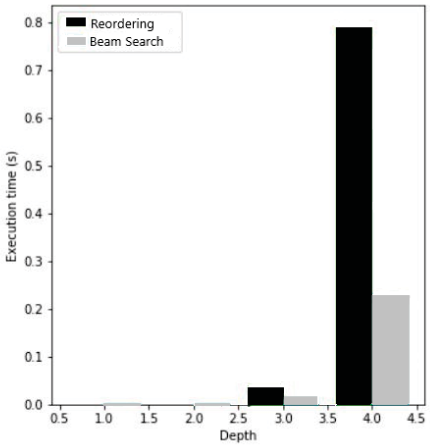

Fig. 4: Execution time of Reordering vs Beam Search

As a matter of course, every thread executes the parallelized segment of code freely using the pragma directive in omp library. The thread assigned to the pruned branch is assigned ”free” state and is allocated back to the pool of forerun created free threads. They can be further be utilised by other segments, thus optimizing the memory usage and execution time. [5]

III. Experimental Results

The first subsection of results computes the relative speedup between the two optimizations used in parallel alpha beta pruning. The optimization which would turn out to be better, will be used as the algorithm for the next section where the architectures used for the implementation have been compared.

A. Reordering vs Beam Search

From Fig4, it can be deduced that average speedup of 2.45 can be obtained relative to the reordering technique by using the beam search optimization. There is a direct implication between the number of visited nodes and the execution time. If less number of nodes are visited, then it will lead to better performance. Therefore, beam search optimizes the number of visited number of nodes when compared to the Reordering optimization as shown in the Fig5.

Thus, we conclude that beam search is the optimal implementation where the speedup is highest and the number of visited nodes is also less, inturn improving performance.

B. Mesh Architecture vs Shared Memory Model

Beam Search with root splitting algorithm is used for implementation on both architectures as the previous section concluded that beam search is the best optimization amongst all. The average speedup achieved by using parallel execution in OpenMP [5] i.e. shared memory model is 3.23 as illustrated in Fig6. The average speedup achieved by using parallel execution in CUDA [8] (using 4 thread model) is 7.26 as illustrated in Fig7. If we compare the implementations of parallel Alpha Beta pruning between OpenMP and

Authorized licensed use limited to: Cornell University Library. Downloaded on May 13,2026 at 21:40:59 UTC from IEEE Xplore. Restrictions apply.

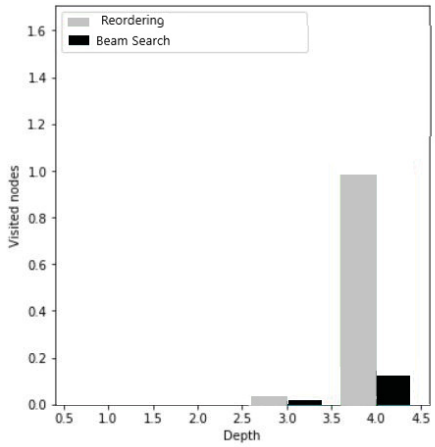

Fig. 5: Number of visited nodes in Beam Search vs Reordering

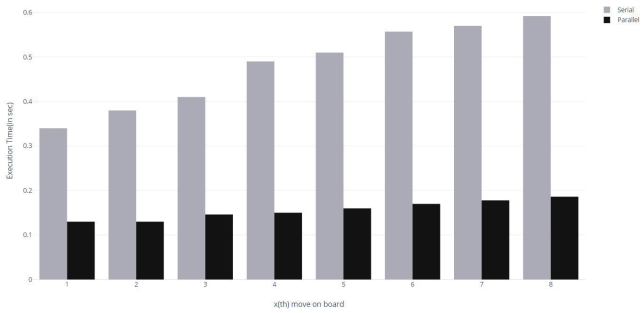

Fig. 6: Parallel (OpenMP) vs sequential Alpha Beta pruning

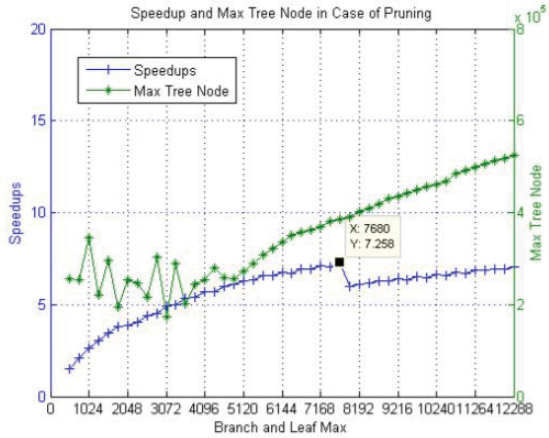

Fig. 7: Parallel (CUDA) vs sequential Alpha Beta pruning

CUDA, CUDA proves to be a better architecture due to possible reasons of efficient multi-processor management and optimized architecture.

IV. Conclusion

The first section of the paper discussed the sequential forms of minimax and Alpha Beta pruning and proved that Alpha Beta pruning can improve performance. Next section of the paper walks through the implementation of parallel Alpha Beta pruning. The algorithm used for the implementation is root split, but this algorithm has been further optimized by using Reordering and beam search. But, different optimization lead to different execution times. Beam Search leads to maximum increase in performance as compared to others. Thus, authors have used beam search for implementation of parallel of Alpha Beta pruning for mesh and shared memory model.

Not every implementation in parallel scenario of Alpha Beta pruning leads to similar speedup. The speedup for Alpha Beta pruning using mesh architecture(CUDA) proves to be almost 2x faster than using OpenMP. So it is concluded that the implementation of Alpha Beta pruning using beam search in CUDA(using 4 thread model) is better than any other possible combination of implementation like Reordering+OpenMP.

References

[1] I. Wikimedia Foundation, "Alphabeta pruning," 26 July 2019. [Online]. Available: https://en.wikipedia.org/wiki/Alpha-beta_pruning

[2] Baeldung, "Introduction to minimax algorithm," November 3, 2018. [Online]. Available: https://www.baeldung.com/java-minimax-algorithm

[3] C. M. M Guidry, "Techniques to parallelize chess." [Online]. Available: https://pdfs.semanticscholar.org/8e1c/9f70aa4849475199e26ccd3e21c662e2d801.pdf

[4] C. wiki, "Alpha-beta," 7 June 2019. [Online]. Available: https://www.chessprogramming.org/Alpha-Beta

[5] S. D. G. V. Akansha Kumari, Shreya Singh, "Parallelization of alpha beta pruning algorithm for enhancing the two player games," vol. 4, pp. 74–81, Feb 2017.

[6] R. S. Kamil Rocki, "Parallel minimax tree searching on gpu," 2010, . (Eds.): PPAM 2009.

[7] N. G. Damjan Strnad, "Parallel alpha-beta algorithm on the gpu," *Journal of Computing and Information Technology*, vol. 19, no. 4, pp. 269–274, 2011.

[8] H. W. T. L. W. L. Liang Li, Hong Liu, "A parallel algorithm for game tree search using gpgpu," *IEEE Transaction on Parallel and Distributed Systems*, vol. 26, pp. 2114 – 2127, July 31, 2014.

[9] H. L. N. Robert M Hyatt, Bruce W Suter, "A parallel alpha/beta tree searching algorithm," *IEEE Transaction on Parallel and Distributed Systems*, vol. 10, pp. 299–308, May 1989, dOI : 10.1016/0167-8191(89)90102-6.

[10] I. Wikimedia Foundation, "Beam search," 5 June 2019. [Online]. Available: https://en.wikipedia.org/wiki/Beam_search

[11] V. Manohararajah, "Parallel alpha-beta search on shared memory multiprocessors," 2011. [Online]. Available: https://pdfs.semanticscholar.org/62b7/2df942c2e26dda4cb25bd5e60dda00687dc4.pdf

[12] Q. F. S. Russ Miller, "Parallel algorithms for regular architectures: Meshes and pyramids," pp. 6–7, January 1996, iSBN : 978-0-262-13233-6, DOI : 10.7551/mitpress/5232.001.0001.

[13] Y. J. Wong., "Shared memory programming openmp," February 2010. [Online]. Available: https://www.mir.wustl.edu/Portals/0/Documents/Uploads/CHPC/WashU_5_openmp.pdf

Authorized licensed use limited to: Cornell University Library. Downloaded on May 13,2026 at 21:40:59 UTC from IEEE Xplore. Restrictions apply.